Power-of-Two-Choices 路由下
慢故障緩解之容量理論極限

DaLaw2

Abstract

慢故障（slow fault）是分散式系統中難以根除的尾延遲違規根源。在慢故障中，節點仍可連線但效能降低。與崩潰故障不同，慢故障能躲過健康檢查，且在高負載下使異常訊號趨於模糊。本文針對 Power-of-Two-Choices（P2C）負載均衡下的慢故障緩解問題，從容量理論角度加以分析，得出三項結論。第一，*操作包絡線*：最大可容忍故障比例為 $f_{\max} = 1 - L/U$，其中 L 為負載因子，U 為利用率上限；此公式無需任何擬合參數，且經實驗驗證。第二，*嚴重度非單調性*：在 P2C 路由下，中度減速（如 10×）導致的尾延遲反而高於極端減速（如 20×），臨界嚴重度為 $s^* = fq/((1-q)(N-f))$。第三，*SHED–P2C 互補性*：P2C 的佇列深度比較與顯式權重削減（SHED）為互補機制，分別作用於路由的不同階段——SHED 在中度嚴重度（2–5×）下可提供 5–70 ms 的 P99 改善，但在 s^* 以上（即 P2C 已自行隔離故障節點的區間）則無需介入。更重要的是，P2C 下只要偵測到故障便降權，效果便與完全隔離幾近相同，證明*偵測速度而非回應策略*才是慢故障緩解的首要瓶頸。透過 15 個故障場景之離散事件模擬，本研究發現：反應式機制（對沖請求、超時重試）對持續性慢故障幾乎無效，而主動偵測搭配流量排除既必要又充分。工程實務者可據此判斷緩解何時可行、哪些嚴重度區間需要介入、以及為何在佇列感知路由下偵測精確度遠不如偵測速度重要。

1 引言

慢故障是分散式系統中最隱蔽的故障類型之一 [4, 5]。在慢故障中，節點仍能回應請求，但處理效能顯著下降。崩潰故障可透過健康檢查立即偵測，慢故障則不然：延遲升高可能源自暫態負載而非真正的效能降級，故障訊號因此模糊不清。若未及時處置，32 節點叢集中的單一慢節點可使系統 P99 延遲增加 3–9×，違反服務等級目標（SLO），並沿服務依賴鏈蔓延至整個系統 [3]。

處理尾延遲的標準做法是*對沖請求*（hedged requests）：等待一小段時間後，將同一請求的副本發送至另一台伺服器，取最先到達的回應 [3]。對沖請求能有效處理*隨機慢請求*——即垃圾回收、上下文切換或背景 I/O 造成的暫態延遲。但持續性慢故障與此截然不同：同一節點在數千個請求中持續緩慢，逐次請求的對沖機制無法解決問題，因為後續請求仍會持續路由至同一故障節點。

現代負載均衡器主要使用 Power-of-Two-Choices（P2C）路由 [7]，依權重隨機抽取兩個候選工作節點，並將請求路由至其中佇列較短者。由此衍生一個核心問題：*P2C 的佇列深度感知是否已能處理慢故障，使顯式緩解成為多餘？*本文以三個理論結果及廣泛的實證加以解答。

1. 操作包絡線：任何緩解策略可容忍的最大故障節點比例為

$$f_{\max} = 1 - \frac{L}{U} \tag{1}$$

其中 L 為系統負載因子，$U \approx 0.89$ 為 M/D/1 排隊延遲開始超線性增長的利用率上限。此公式毋須擬合參數，即能預測四個負載等級的成功／失敗邊界（第 4.1 節）。

2. 嚴重度非單調性：在 P2C 路由下，「最糟」的減速因子並非最高值，而是某個特定值。在此減速因子下，到達故障節點的流量比例恰好等於目標百分位的尾端寬度：

$$s^* = \frac{f \cdot q}{(1-q)(N-f)} \tag{2}$$

高於 s^* 時，P2C 透過佇列深度比較自動將流量導離故障節點，系統無需顯式介入即可*自行隔離*故障節點（第 4.2 節）。

3. **SHED–P2C** 互補性：P2C 路由與顯式權重削減（SHED）作用於路由決策的不同階段：SHED 控制*候選者被選取的機率*，而 P2C 提供*佇列深度的決勝機制*。兩者互補而非冗餘。SHED 的效益隨 $O(N^{-\alpha})$（$\alpha \approx 1.3$）衰減，在中度嚴重度（2–5×）時最大化，而在 s^* 以上（P2C 自行隔離之處）則趨於消失。關鍵的是，在 P2C 下最低權重 $w_{\min} = 0.05$ 可達到與完全隔離（$w = 0$）幾乎相同的 P99，證明*偵測精確度遠不如偵測速度重要*（第 4.3 節）。

上述結果經 8 至 64 節點叢集的離散事件模擬加以驗證，涵蓋 15 個故障場景。評估比較八種緩解策略，包括三種文獻基線（對沖請求 [3]、異常值黑名單 [6] 和超時重試）。主要實證發現如下：

- 偵測速度是首要瓶頸：在 P2C 路由下，漸進式權重削減與完全隔離產生幾乎相同的 P99 延遲（差距 $< 2\,\text{ms}$）。一旦偵測到故障節點，回應策略便無關緊要。此結論在不同叢集大小、嚴重度等級與負載因子下皆成立（第 6.4 節）。
- 反應式機制失效：對沖請求和超時重試與無緩解的情況相比，SLO 達成率僅提升不到 0.3 個百分點——它們缺乏偵測機制，是為隨機慢請求而非持續故障設計的（第 6.2 節）。
- 主動偵測既必要又充分：所有能偵測故障節點並重新導向流量的策略，在 80% 負載下，於所有場景中均達成 $\text{P99} < 45\,\text{ms}$。無緩解與任何主動策略的差距（5–70 ms）遠超不同主動策略之間的差距（第 6.2 節）。

本文的實務貢獻如下：(1) 判斷給定負載與故障比例下緩解是否可行的簡明公式；(2) 識別最危險減速區間的分析工具；(3) 透過互補性定理釐清何時 P2C 本身即已足夠、何時需要顯式偵測；(4) 證明偵測速度（而非回應精確度）才是慢故障緩解系統首要設計目標的實證依據。

2 背景與動機

2.1 生產系統中的慢故障

灰色故障（gray failure）[5] 泛指各種部分故障，其中元件並未完全停機，卻也無法正常運作。慢故障是灰色故障的一個子類，其特徵為服務時間延長至 s 倍（$s > 1$）。常見成因包括：記憶體壓力導致過度垃圾回收、磁碟退化增加 I/O 延遲、溫度限流降低 CPU 頻率，或特定路徑上的網路擁塞 [4]。

與崩潰故障不同，慢故障難以偵測，因為受影響的節點仍持續回應，僅是速度較慢。標準健康檢查（TCP 探測、心跳）無法區分減速達 5 倍的節點與暫態負載下的健康節點。故障訊號的模糊問題在高系統利用率下尤為明顯，此時*所有*節點都會因排隊效應而延遲升高。

2.2 Power-of-Two-Choices 負載均衡

Power-of-Two-Choices（P2C）[7] 是現代分散式系統中的主流負載均衡演算法，Envoy [1]、nginx 及眾多雲原生基礎設施均採用此演算法。其運作方式為：每當請求到達時，P2C 隨機抽樣兩個候選工作節點，並將請求路由至佇列較短的那個。

P2C 對慢故障有一種隱式緩解效果，雖然重要，一般討論卻鮮少提及。當某節點的減速因子為 s 時，其處理速度降低 s 倍，佇列隨之增長。P2C 比較佇列深度時便傾向於避開該節點，將流量導向較健康的節點。第 4.2 節將展示，此隱式緩解在極端嚴重度（$s > s^*$）下效果顯著，但在中度嚴重度（$s < s^*$）下則不足，而後者正是顯式緩解策略能發揮作用的區間。

2.3 現有緩解方法

對沖請求

Dean 與 Barroso [3] 提出一種緩解尾延遲的策略：等待一小段時間（通常為預期服務時間）後，將同一請求的副本發送至另一台伺服器，客戶端採用先到達的回應。此方法能有效處理隨機性慢請求，然而如本文後續分析所示，對持續性慢故障則無效，因為後續請求仍會持續路由至同一故障節點。

異常值黑名單

Cassandra 的動態偵測器（dynamic snitch）[6] 追蹤各節點延遲，並將延遲超出叢集中位數達 k 個標準差以上的節點臨時加入黑名單。此為前瞻式機制，能將後續流量重新導向，但在高負載下存在振盪風險：將節點加入黑名單實質上等於減少系統可用容量，進而增加剩餘節點的負擔。

斷路器

Istio 與 Envoy 內建斷路器機制，當錯誤率或延遲超過閾值時觸發，將流量從不健康的端點重新導向。此類機制雖然有效，但需手動調整閾值，且無法自適應於不同的負載條件。

2.4 緩解陷阱

推動本研究的關鍵動機之一，是*緩解陷阱*（mitigation trap）：在高系統負載下，樸素的緩解策略*反而可能適得其反*。考慮 85% 負載下的對沖請求：每次對沖產生一個額外請求，使健康節點的利用率從 $\rho = 0.85$ 推向飽和。M/D/1 模型的排隊延遲依 $\rho/(2(1-\rho))$ 遞增，形成正回饋迴圈：對沖增加負載，負載增加延遲，延遲觸發更多對沖。

實驗結果驗證了此現象（第 6.2 節），並據此歸納出本文負載感知自適應框架的核心設計原則：任何緩解策略皆須將自身的容量成本納入考量。

3 系統模型

3.1 架構

本文建模一個單層分散式服務：N 個同質工作節點組成叢集，前端設有一個負載均衡器。請求依泊松過程抵達系統，總到達速率為 $\Lambda = L \cdot N \cdot \mu$，其中 $L \in (0,1)$ 為*負載因子*，$\mu = 1/t_{\text{base}}$ 為單一節點之服務速率。各工作節點以先進先出方式逐一處理請求（單伺服器模型）。正常情況下服務時間為定值 t_{base}，整體符合 M/D/1 佇列模型。

負載均衡器採用加權 Power-of-Two-Choices（P2C）策略：針對每筆請求，依權重 $w_i \in [0,1]$ 按比例隨機抽取兩個候選節點，再將請求路由至其中佇列較短者。

3.2 故障模型

當節點 i 發生*慢故障*時，其服務時間將乘以減速因子 $s_i > 1$：

$$t_i = s_i \cdot t_{\text{base}}$$

假設 N 個節點中同時有 f 個發生故障，且故障具有*持續性*：一旦發生，減速效果將延續至系統主動恢復為止（若有恢復機制）。本文探討以下五種故障模式：

- 永久性：故障發生後減速因子 s 維持恆定。

- 漸進式：$s(t) = 1 + (s_{\max} - 1)(1 - e^{-\beta t})$，減速隨時間逐漸加劇。
- 波動式：在 $s = s_{\text{peak}}$ 和 $s = 1$ 之間交替，具有可配置的開／關持續時間。
- 級聯式：故障依交錯的時間間隔先後擴散至不同節點。
- 恢復式：故障活躍一段固定時間後，節點恢復正常。

3.3 效能指標

本文以下列尾端延遲指標評估系統效能：

- **P99/P999** 延遲：請求端到端延遲之第 99 及第 99.9 百分位（含排隊等候時間）。
- **SLO** 違反率：延遲超過 SLO 閾值（實驗中設為 50 ms）之請求佔比。
- 有效吞吐量（goodput）：每秒在 SLO 內完成的請求數。
- 受影響比例：經由故障節點處理之請求佔比，用以衡量負載均衡器的分流效果。

3.4 極小極大遺憾

為比較各策略在多種場景下的效能，本文採用*極小極大遺憾*（minimax regret）準則。給定策略 j 與場景 i，其遺憾定義為：

$$R_{ij} = P99_{ij} - \min_k P99_{ik}$$

策略 j 的極小極大遺憾為 $\max_i R_{ij}$，即該策略在最不利場景下相對於最優策略的效能落差。極小極大遺憾愈低，代表策略在各種運行條件下愈具穩健性。

4 分析結果

本節推導三個解析結果，刻畫 P2C 路由下慢故障緩解的根本限制與機制。

4.1 操作包絡線

當偵測器將故障節點隔離時，剩餘健康節點的負載隨之上升。直覺上，存在一個臨界點——隔離太多節點，健康節點本身就會過載。以下定理量化了這個臨界點。

定理 1 (操作包絡線)**.** *在具有負載因子 L 的 N 節點叢集中，以隔離為手段的緩解策略，所能容忍的最大故障節點比例為：*

$$f_{\max} = 1 - \frac{L}{U} \tag{3}$$

其中 U 為使單節點排隊延遲達到不可接受程度的利用率上限。

Proof. 當 f 個節點被隔離（從活躍池中移除）時，剩餘 $N - f$ 個健康節點必須承擔全部系統負載。每節點的有效利用率變為：

$$\rho_{\text{eff}} = \frac{\Lambda}{(N - f) \cdot \mu} = \frac{L \cdot N}{N - f} \tag{4}$$

為使系統在可接受的排隊延遲下維持穩定，需滿足 $\rho_{\text{eff}} \le U$。對 f 求解可得：

$$f \le N \left(1 - \frac{L}{U}\right) \tag{5}$$

□

Table 1: 不同服務時間分布下以平均延遲為基準的利用率上限 U 與操作包絡線 $f_{\max}$（$L = 0.80$）。

分布	C_s^2	U	$f_{\max}$	最大故障數（N=32）
M/D/1	0	0.889	10.0%	3.2
M/G/1（溫和）	0.25	0.865	7.5%	2.4
M/M/1	1.0	0.800	0.0%	0.0

簡言之，L/U 是「最少需要多少比例的節點才能撐住負載」，超出的部分才是可隔離的餘裕。負載越高，餘裕越小。

定理 1中的 U 是一個自由參數——它取決於對「可接受排隊延遲」的定義。接下來從 M/D/1 模型推導其封閉形式，如此一來，包絡線便成為無需調參的預測工具。對於服務時間為確定值 t_{base} 的單伺服器佇列，平均逗留時間為：

$$T(\rho) = t_{\text{base}} \cdot \frac{2-\rho}{2(1-\rho)} \tag{6}$$

推論 **2** (封閉式利用率上限)**.** *考慮一 M/D/1 佇列，其基底服務時間為 t_{base}，SLO 目標以平均延遲衡量，則利用率上限為：*

$$U = 1 - \frac{t_{base}}{2 \cdot SLO - t_{base}} = \frac{2(SLO - t_{base})}{2 \cdot SLO - t_{base}} \tag{7}$$

Proof. 令公式 6中 $T(U) = \text{SLO}$，設 $\alpha = 2\,\text{SLO}/t_{\text{base}}$，直接求解一元一次方程式可得 $U = (\alpha-2)/(\alpha-1) = 1 - t_{\text{base}}/(2 \cdot \text{SLO} - t_{\text{base}})$。 □

以本文參數（$\text{SLO} = 50\,\text{ms}$，$t_{\text{base}} = 10\,\text{ms}$）：$U = 1 - 10/90 = 8/9 \approx 0.889$。全文使用 $U = 8/9$。

推廣至 M/G/1 服務時間

以下推廣並非主線論述所需，但可量化服務時間變異性對包絡線的影響。當服務時間具有變異係數 C_s 時，Pollaczek–Khinchine 公式給出 $T(\rho) = t_{\text{base}}[1 + (1 + C_s^2)\,\rho/(2(1-\rho))]$。令 $T(U) = \text{SLO}$ 求解可得推廣上限：

$$U(C_s) = \frac{\alpha}{\alpha + 1 + C_s^2} \quad \text{其中} \alpha = 2\left(\frac{\text{SLO}}{t_{\text{base}}} - 1\right) \tag{8}$$

表 1 展示服務時間變異性的影響。

M/M/1（$C_s^2 = 1$）的結果值得特別關注：$U = 0.800$，意味著在 $L = 0.80$ 時故障容忍度為零——任何慢故障都會立即違反 SLO。此結果量化了一個直覺認識——服務時間變異性會侵蝕系統餘裕，也表明確定性 M/D/1 假設提供了故障容忍度的樂觀上界。

備註 **3** (平均值上限的有效性與緊密性)**.** *表 1 由平均延遲導出 U，但 SLO 通常定義於 P99。對 M/D/1（$C_s^2 = 0$），差距很小：確定性服務時間使回應時間的變異很小，故 $U_{P99} \gtrsim U_{mean}$，公式為保守的（安全的）。對 M/M/1（$C_s^2 = 1$），差距很大：模擬顯示 P99 在 $L \approx 0.70$ 即超越 50 ms SLO，基於平均值的 U 對高變異分布而言是樂觀的。就緊密性而言，在 $L = 0.80$、$N = 32$ 時理論上限為 3.2 個節點，而實驗中隔離 4 個節點仍可達成 P99 $< 50\,ms$，此餘裕源自平均延遲與 P99 之間的差距。包絡線略為保守，但預測的通過／失敗邊界與實驗相差不超過一至兩個節點。*

Table 2: 操作包絡線驗證。理論預測最大故障比例；實驗欄位顯示自適應策略是否滿足 P99 $< 50\,\text{ms}$ 之 SLO。

L	$f_{\max}$	$f_{\max}$ 節點數	實驗邊界	吻合
0.70	21.3%	6.8	8 通過, 10 失敗	✓†
0.80	10.0%	3.2	4 通過, 6 失敗	✓
0.85	4.4%	1.4	2 通過, 4 失敗	✓†
0.90	< 0	—	0 通過, 2 失敗	✓

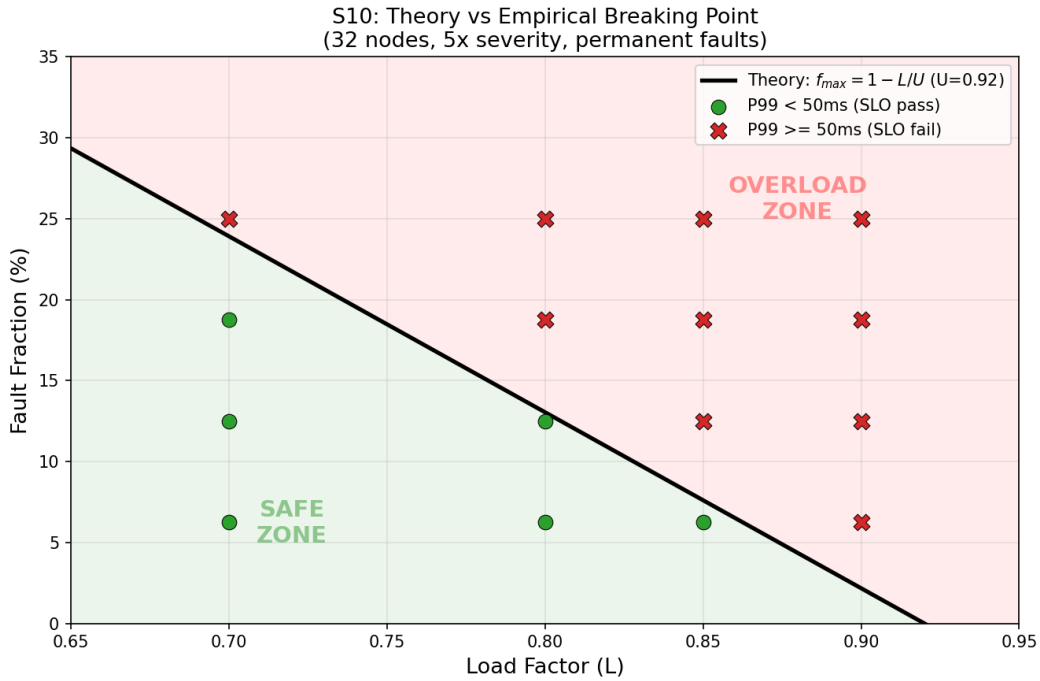

Figure 1: 操作包絡線：理論與實驗對照。邊界 $f_{\max} = 1 - L/U$（虛線）劃分安全區（P99 $< 50\,\text{ms}$）與過載區，驗證過程無需任何擬合參數。

非隔離策略

定理 1 中的上界假設故障節點被完全隔離。若策略僅*削減*故障節點流量而非完全隔離，則故障節點貢獻 μ/s_i 的有效服務率，有效利用率為：

$$\rho_{\text{eff}} = \frac{L \cdot N}{(N - f) + \sum_{i \in F} 1/s_i} \tag{9}$$

其中 F 為故障節點集合。此值始終低於完全隔離的情況，證實操作包絡線為*保守估計*。

以 $N = 32$、永久 5× 故障的配置，在四個負載等級下驗證定理 1（表 2）。

該公式無需任何擬合參數即可預測通過／失敗邊界。標記 † 的列中，實驗通過的故障數超過理論上限（如 $L = 0.70$ 時理論上限為 6.8 個節點，但 8 個仍通過），這是因為 U 由*平均*延遲導出，而 P99 的有效上限略高（見備註 3），使包絡線成為保守估計。$L = 0.90$ 超過 $U = 8/9$，即使無故障亦無法滿足 SLO。

圖 1 以圖形呈現各負載等級下的操作包絡線，展示安全區與過載區之間的明確邊界。

4.2 嚴重度非單調性

假設 **1** (平均場 P2C 近似)**.** *在 P2C 加權隨機抽樣下，穩態時各節點所接收的流量與其有效服務容量成正比。即容量為 μ_i 的節點接收總流量的 $\mu_i/\sum_j \mu_j$ 份額。*

當節點數量夠大且抽樣按容量加權時，此近似趨於精確。對有限叢集而言，P2C 的佇列深度比較會使故障節點的流量分配產生輕微的正偏差（見表 3：$f = 4$ 時低估 10–14%，$f = 8$ 時 $< 1\%$）。

定理 **4** (臨界嚴重度)**.** *在假設 1 下，流量到達 f 個（減速因子為 s）故障節點的比例為：*

$$p_{fault}(f, s, N) = \frac{f}{s \cdot (N - f) + f} \tag{10}$$

使故障節點的延遲影響在第 q 百分位上不可見的臨界嚴重度為：

$$s^* = \frac{f \cdot q}{(1 - q)(N - f)} \tag{11}$$

換言之：P2C 路由會讓越嚴重的故障節點分到越少的流量。當嚴重度夠高時，故障流量的佔比降到 $1 - q$ 以下——此時故障請求在第 q 百分位上統計不可見。s^* 就是這個臨界點。

Proof. 在 P2C 下，流量按有效服務容量的比例分配。健康節點容量為 μ；故障節點容量為 μ/s。故障節點的流量佔比化簡為 $p_{\text{fault}} = f/[s(N-f) + f]$（公式 10）。系統延遲服從混合分布，當 $p_{\text{fault}} > 1 - q$ 時故障請求在第 q 百分位可見。令 $p_{\text{fault}} = 1 - q$ 並對 s 求解即得 s^*。 □

非單調性機制

當 $s < s^*$ 時，$p_{\text{fault}} > 1 - q$，故障節點的延遲主導第 q 百分位。系統 P_q 隨 s 增大而升高，因故障節點的服務時間為 $s \cdot t_{\text{base}}$。當 $s > s^*$ 時，$p_{\text{fault}} < 1 - q$，故障請求在第 q 百分位統計上不可見。系統 P_q 回落至健康節點值。因此，最差的 P_q 發生在 $s = s^*$ 附近，而非 $s \to \infty$。此即嚴重度非單調性：將故障嚴重度提高至 s^* 以上，尾延遲反而降低。

換言之，系統管理員面臨的最棘手情境並非極端故障（$s \gg s^*$），而是中度故障（$s \approx s^*$）——這恰恰是最難偵測的區間。

百分位相關風險

同一故障在不同百分位上的可見性可能截然不同，此即百分位相關風險。對 $f = 4$，$N = 32$：$s^*_{P99} = 14.1$ 但 $s^*_{P999} = 142.7$。20× 減速對 P99 不可見（$20 > 14.1$）但對 P999 可見（$20 < 142.7$），這解釋了實驗中觀察到的現象：P99 = 49 ms（正常）但 P999 = 17,427 ms（災難性）。

表 3 驗證上述流量分配模型。

模型在 $f = 4$ 時系統性低估 10–14%（因 P2C 的佇列深度比較機制使較慢節點接收到略多於模型預測的流量），而在 $f = 8$ 時則精確至三位有效數字。$f = 4$ 時的系統性正偏差（比率 ≈ 1.10–1.14）源自 P2C 佇列深度比較中的平手打破機制，此殘餘偏差以 $O(1/N)$ 的速率遞減。就實際用途而言，平均場近似足以預測臨界嚴重度 s^* 及 P99 可見性邊界。

圖 2 展示不同故障數量及減速因子組合下的嚴重度非單調性。熱力圖清楚顯示中度嚴重度（如 4 個故障 10×：P99 = 439 ms）產生的尾延遲比極端嚴重度（如 4 個故障 20×：P99 = 49 ms）更差。

4.3 SHED–P2C 互補性

操作包絡線與臨界嚴重度界定了緩解的可能範圍。接下來分析 P2C 與顯式權重削減（SHED）之間的互動，證明兩者為互補機制——而非冗餘。

Table 3: 到達故障節點的流量比例：理論與實驗對照（無緩解，$N = 32$，$L = 0.80$）。

f	s	理論 p_{fault}	實驗值	比率
4	3×	4.55%	5.20%	1.14
4	5×	2.78%	3.12%	1.12
4	10×	1.41%	1.56%	1.11
4	20×	0.71%	0.78%	1.10
8	3×	10.00%	10.42%	1.04
8	5×	6.25%	6.26%	1.00
8	10×	3.23%	3.23%	1.00
8	20×	1.64%	1.64%	1.00

P2C 下的兩階段路由

P2C 路由做出兩個依序的決策：

1. *候選者選取*：依權重 w_i 的比例抽取兩個工作節點。故障節點被選為候選者的機率約為 $2w_i/\sum_j w_j$。
2. *佇列深度比較*：在兩個候選者中，將請求路由至佇列較短的一方。

SHED 作用於第一階段，降低故障節點的 w_i；P2C 的隱式迴避則作用於第二階段，透過佇列深度比較實現。因此，第二階段只能在第一階段所選定的候選者之間做決策。

命題 **5** (SHED–P2C 互補性)**.** *在採用 $d = 2$ 選擇的 P2C 路由下，考慮一個 N 節點叢集中的單一故障節點（嚴重度為 s），令 $\Delta_{SHED}(N, s)$ 為對其施加 SHED 權重削減後的 P99 改善量。則：*

(a) 規模衰減：$\Delta_{SHED}(N, s) = O(N^{-\alpha})$，*其中* $\alpha \approx 1.3$（*中度嚴重度區間*）。

(b) 嚴重度區間：Δ_{SHED} *在中等* s（*通常* 3–5×）*時最大化，在* $s < 1.5$ *及* $s > s^*$ *時可忽略。*

(c) 實務等價：*在 P2C 下，最低權重* $w_{\min} = 0.05$ *可達到與完全隔離*（$w = 0$）*相差* $O(w_{\min}/N)$ *以內的 P99。*

直覺上：P2C 的佇列比較已是一道強大的防線。SHED 在此之上再加一層過濾，但因為第一道防線已經很有效，第二道的邊際效益隨叢集規模衰減。

此結論源自兩階段結構。無 SHED（均勻權重）時，每次請求選到故障節點作為候選者的機率約為 $2/N$。在 SHED 權重 $w_{\min}$ 下，此機率降為 $\approx 2w_{\min}/N$——候選率降低了 $1/w_{\min}$ 倍。然而，P2C 的第二階段透過佇列比較已在多數時候拒絕故障節點。到達故障節點的淨流量在無 SHED 時為 $O(1/N)$，有 SHED 時為 $O(w_{\min}/N)$。隨著 N 增長，故障節點的基準流量已可忽略，SHED 的邊際效益因而消失。

容量損失邊界

上述互補性有一個重要的例外：當健康節點的有效負載 $L_{\text{healthy}} = L \cdot N/(N - f_{\text{shed}})$ 超過排隊膝點（≈ 0.88–0.90）時，SHED 的權重削減反而有害——重新分配的流量使健康節點飽和，導致的排隊延遲增加超過原先的改善。

表 4 於不同叢集大小、嚴重度等級與故障數下驗證上述互補性命題。

Figure 2: 嚴重度非單調性熱力圖。$L = 0.80$，$N = 32$ 下，按故障數量與減速因子呈現 P99 延遲（ms）。非單調模式清晰可見：在每一種故障數量下，存在一個臨界嚴重度，超過該值後 P99 急遽下降。

最後一列表明 SHED 本身*並非*無效——在無佇列感知路由下可帶來 15,697 ms 的改善。P2C 的佇列深度比較大幅壓縮了 SHED 的邊際貢獻。此結果證實兩種機制互補：P2C 提供主要防禦，SHED 在候選者選取階段提供次級過濾。

在 P2C 下，SHED 與 ISO 的 P99 差距始終 < 2 ms（表 4），而無緩解與任何主動策略的差距則為 5–70 ms。此一數量級的差異意味著，工程上的首要挑戰是多快偵測到故障節點，而非多*精確*地調降其權重。在 P2C 下，粗略的二元偵測器（「降權或不降權」）即可達到近乎最優的效果。

4.4 對緩解設計的啟示

上述公式將任意 (f, s, L, N) 配置劃分為三個區域：

1. 不可行（$f/N > f_{\max}$）：任何策略都無法維持 SLO。容量理論極限不可逾越。
2. P2C 充分（$f/N \leq f_{\max}$ 且 $s > s^*$）：P2C 透過隱式佇列深度迴避自動處理故障節點。顯式緩解非必要，反而可能引入額外開銷。
3. 需要緩解（$f/N \leq f_{\max}$ 且 $s \leq s^*$）：故障比例在容量限制內，但 P2C 無法充分避開故障節點。需要主動偵測與流量重導。在 P2C 下，漸進式權重削減與完全隔離的差異可忽略（< 2 ms）——關鍵在於偵測速度。

Table 4: SHED 在 P2C 下相對於無緩解的 P99 改善（ms）。SHED 將權重降至 $w = \max(0.05, 1 - \sigma)$；ISO 設 $w = 0$。所有數值為 10 次執行的平均值。

配置	no_mit	SHED	ISO	SHED Δ	備註
規模掃描（$s = 3\times$，$L = 0.70$）					
$N = 8$	79.7	34.6	34.3	−45.1	小 N 時效益大
$N = 16$	59.3	29.8	28.2	−29.5	
$N = 32$	46.6	27.8	26.9	−18.8	
$N = 64$	28.5	26.8	26.4	−1.7	大 N 時可忽略
嚴重度掃描（$N = 16$，$L = 0.70$）					
$s = 1.5$	29.3	28.0	28.1	−1.3	低於偵測閾值
$s = 3$	59.3	29.8	28.2	−29.5	效益峰值
$s = 5$	99.7	29.7	28.2	−70.0	接近 s^*
$s = 10$	34.6	34.6	34.6	+0.0	高於 s^*：已自行隔離
無佇列感知路由（加權隨機）					
$s = 3$, WR	17757	2060	—	−15697	無 P2C 時 SHED 至關重要

結合互補性命題（第 4.3 節），此框架釐清顯式緩解的適用條件：在區域 2，P2C 自行隔離故障節點，無需偵測或緩解——但監控 P999 至關重要，因高嚴重度故障在更深的百分位仍然可見。在區域 1，任何策略皆無濟於事。

5 參考實作

為驗證上述分析，本文實作 Prism——一套用於偵測故障節點並降低其路由權重的框架。如第 4.3 節所示，在 P2C 下，偵測速度遠比具體的回應策略重要。以下對 Prism 的說明旨在描述評估中所採用的偵測機制。

5.1 架構概覽

Prism 運行於控制平面，與 P2C 資料平面並行。每個*決策週期* Δ（預設 0.5 s），執行三階段管線：*監測* → *偵測* → *行動*。

在監測階段，系統從資料平面收集各節點指標：請求離開間隔、佇列深度及全系統吞吐量。將理論最大吞吐量與實測吞吐量之差定義為*剩餘容量*，並以指數移動平均（$\alpha = 0.3$）加以平滑。

在偵測階段，以離開間隔的 P10 百分位估計各節點服務時間。根據 M/D/1 佇列理論，此訊號不受負載影響（詳見第 5.2 節）。嚴重度定義為：

$$\sigma_i = \max\left(0,\ 1 - \frac{t_{\text{base}}}{d_i}\right) \tag{12}$$

其中 d_i 為節點 i 的估計服務時間。當減速為 5 倍時 $\sigma = 0.8$；減速為 10 倍時 $\sigma = 0.9$。

在行動階段，根據嚴重度閾值選擇各節點策略，並施加相應的流量調整。

5.2 偵測：離開間隔估計

偵測慢故障的核心難題在於區分真正的服務退化與暫態排隊效應。在高負載（$\rho > 0.85$）下，所有節點的延遲均會升高，使延遲式偵測器產生誤報。

本框架利用 M/D/1 佇列的一項性質：當佇列持續處於忙碌狀態時（高利用率下多數時段皆如此），離開間隔即為實際服務時間，不受佇列深度影響。取離開間隔的 P10 百分位可濾除佇列清空時的空閒間隙，從而逼近服務時間的下界。

此訊號具*負載不變性*：基底服務時間 $t_{\text{base}} = 10\,\text{ms}$ 的節點，無論系統負載為 70% 或 95%，均產生 $d \approx 10\,\text{ms}$。5 倍減速時，在任何負載水準下皆得 $d \approx 50\,\text{ms}$。

為避免暫態噪聲誤觸緩解機制，本框架要求*持續確認*：嚴重度須連續兩個決策週期超過最低閾值（$\sigma > 0.05$），才會將該節點標記為故障。

5.3 回應策略

如第 4.3 節所示，在 P2C 下具體的回應策略對 P99 的影響可忽略（$< 2\,\text{ms}$）。本實作依嚴重度閾值（$\theta_{\text{spec}} < \theta_{\text{shed}} < \theta_{\text{iso}}$）選擇三種策略：輕度退化時對沖（向另一健康節點發送副本），中度退化時削減（依嚴重度降低權重 $w_i = \max(w_{\min}, 1 - \sigma_i \cdot \text{spare_factor})$），嚴重退化時隔離（完全移除）。隔離受容量邊界約束：僅當隔離後剩餘有效利用率低於 U 時方允許執行。

5.4 負載感知閘控

本框架的核心設計原則為：*每項緩解行動皆有容量成本*。對沖使請求負載加倍；削減會將負載轉移至健康節點；隔離則完全移除容量。當系統負載偏高時，其成本可能超過效益，反而形成緩解陷阱。

Prism 以剩餘容量作為統一的閘控訊號，同時管控三種策略：

- 對沖預算 $\propto$ 剩餘容量（剩餘容量為零時不進行對沖）。
- 權重削減幅度 $\propto$ spare_factor （剩餘容量 < 0.08 時不削減）。
- 當隔離後有效利用率 $\rho_{\text{eff}} > U$ 時，阻止隔離。

如此可確保 Prism 優雅降級：低負載時積極隔離以加速恢復；高負載時保守削減以保留容量；過載時則完全退避，避免適得其反。

6 評估

本節以 SimPy 離散事件模擬驗證上述分析結果。目標為：(1) 驗證操作包絡線、嚴重度非單調性與 SHED–P2C 互補性定理；(2) 與文獻基線比較；(3) 分析不同運作條件下的穩健性。

6.1 實驗設定

叢集由 $N = 32$ 個工作節點組成，各以 M/D/1 佇列建模，$t_{\text{base}} = 10\,\text{ms}$。預設負載因子 $L = 0.80$（總到達率 $\Lambda = 2{,}560$ req/s），SLO 目標為 P99 $\leq 50\,\text{ms}$。

比較對象為以下八種策略：

- `no_mitigation`：不回應慢故障（下界）。
- `fixed_speculation`：始終對沖，從不削減或隔離。
- `fixed_shedding`：始終透過降低權重來削減負載。
- `fixed_isolation`：激進地隔離慢節點（$\theta_{\text{iso}} = 0.3$）。
- `adaptive`（Prism）：本文提出之自適應多策略框架。

Table 5: 主要故障場景配置。

編號	模式	故障數	嚴重度	持續時間
S2	漸進式	2	1→15×	60 s
S3	閃群	2	3× + 負載尖峰	60 s
S4	多節點	4	2–5×（隨機）	60 s
S5	波動式	2	5× 開/關	60 s
S6	級聯	1→2→3	5/5/3×	90 s
S7	恢復	2	8× 後恢復	100 s

Table 6: 各場景在 $L = 0.80$ 下的 P99 延遲（ms）。粗體標示各場景之最佳值。

基線	S2	S3	S4	S5	S6	S7
PRISM	**30.0**	39.3	43.8	**30.3**	**32.8**	**29.7**
lit_blacklist	30.5	41.1	**37.4**	30.5	**32.8**	30.5
fixed_iso	30.3	39.9	37.6	30.6	**32.8**	**29.7**
fixed_spec	31.9	41.0	54.6	31.2	35.0	29.8
fixed_shed	33.6	**39.2**	69.3	31.1	37.0	29.9
lit_hedged	61.3	68.8	110.9	34.5	110.6	32.3
lit_retry	61.3	68.7	108.8	34.5	110.2	32.3
no_mitig	67.2	73.3	125.1	34.6	125.7	32.3

- `lit_hedged`：無條件對沖請求 [3]，延遲閾值 $= 3 \cdot t_{\text{base}}$，10% 預算上限。
- `lit_blacklist`：異常值黑名單（Cassandra 風格 [6]），$k = 2.0$，冷卻週期 3 期。
- `lit_retry`：超時重試，逾時閾值 $= 0.7 \times \text{SLO}$。

表 5 總結 6 個主要場景（S2–S7），各測試一種不同的故障模式。每個場景以不同隨機種子執行 10 次；S4 因隨機減速因子導致變異較高，故執行 20 次。

6.2 反應式與主動式策略

表 6 呈現所有場景的 P99 延遲，結果顯示三個明顯的效能層級。

第一層為主動策略（P99 < 45 ms）。PRISM、lit_blacklist 和 fixed_isolation 在每個場景均達成 P99 低於 45 ms。三者的核心機制相同：先偵測故障節點，再從路由中移除——偵測方法本身（離開間隔、延遲異常值或固定閾值）並非決定性因素。

第二層為反應式策略（P99 接近無緩解基線）。lit_hedged 與 lit_retry 相較於無緩解僅有微幅改善：SLO 違反率 0.7–3.4% vs. 0.8–3.7%（改善 < 0.3 個百分點）。根本原因在於：對沖機制係針對隨機性慢請求所設計，不適用於持續性故障。各請求獨立決定是否對沖，但後續請求仍然會路由至同一故障節點。

第三層為無緩解基線，P99 介於 32–125 ms 之間，視場景嚴重度而定。

圖 3 以長條圖呈現八種策略的效能分層，主動式與反應式策略之間的顯著效能差距一目瞭然。

6.3 標準負載下的極小極大遺憾

表 7 呈現 S2–S7 的極小極大遺憾分析。

在標準負載（$L = 0.80$）下，三種主動策略——lit_blacklist（0.6 ms）、fixed_isolation（0.2 ms）與 PRISM（0.2 ms）——皆達到近乎零的極小極大遺憾。此結果證實了互補性定理的核心預測：在 P2C 下，能否偵測到故障遠比選用何種回應策略重要。

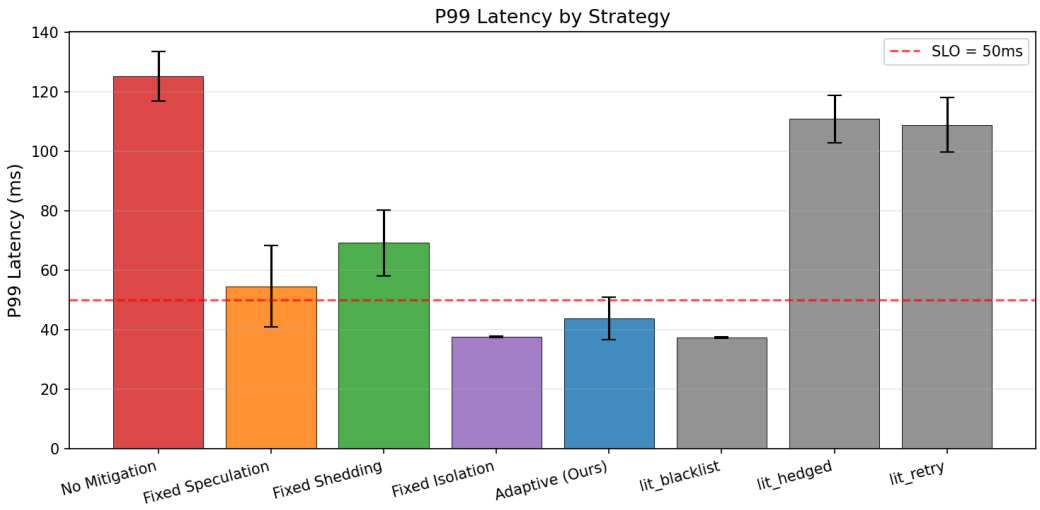

Figure 3: 八個基線的 P99 延遲比較（S4 場景，$L = 0.80$）。三個效能層級清晰可見：主動策略（P99 $< 45\,\text{ms}$）、反應式策略（P99 $\approx$ no_mitigation）和無緩解基線。

Table 7: $L = 0.80$ 下的極小極大遺憾（ms，高於場景最佳 P99）。

策略	最大遺憾	平均遺憾	最差場景
lit_blacklist	0.6	0.1	S7
fixed_iso	0.2	0.1	S4
PRISM	0.2	0.1	S4
fixed_spec	9.5	2.2	S4
fixed_shed	22.2	5.0	S4
lit_retry	76.4	36.2	S6
lit_hedged	76.9	36.4	S6
no_mitig	92.2	42.0	S6

6.4 SHED–P2C 互補性驗證

為驗證互補性定理（第 4.3 節），本節比較純 P2C（無緩解）、SHED（權重削減但不隔離）與完全隔離三者的表現。比較涵蓋四個維度：叢集大小、嚴重度、負載與故障數（S15，每種配置 10 次執行）。

規模相依性

圖 4 證實 $O(N^{-\alpha})$ 的衰減規律。SHED 相對於無緩解的 P99 改善從 $N = 8$ 時的 45.1 ms 降至 $N = 64$ 時的 1.7 ms（1 個故障，3×，$L = 0.70$）。在 $N = 64$ 時，純 P2C 幾乎可匹敵顯式緩解，與 P2C 的候選率 $\approx 2/N \approx 3\%$ 一致。

自行隔離閾值

在 $s = 10$（$N = 16$，$L = 0.70$）時，所有策略收斂至 34.6 ms：SHED、隔離和無緩解產生相同的 P99。此結果證實，在 s^* 以上，P2C 的佇列深度訊號已足以實現完全的隱式隔離。非單調性十分明顯：P99 從 $s = 1.5$ 的 29.3 ms 升至 $s = 5$ 的 99.7 ms，再降至 $s = 10$ 的 34.6 ms。

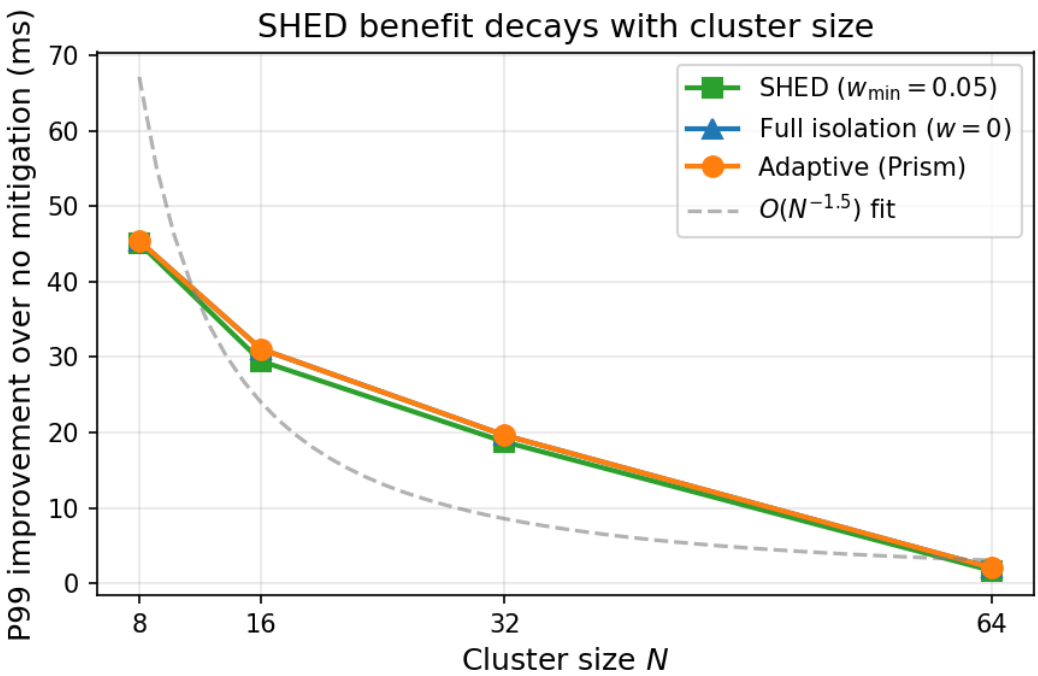

Figure 4: SHED 效益隨叢集規模以 $O(N^{-\alpha})$（$\alpha \approx 1.3$）衰減。三種主動策略（SHED、隔離、自適應）在每個 N 值下產生幾乎相同的 P99 改善，證實策略選擇無關緊要。在 $N = 64$ 時，效益縮小至 $< 2\,\text{ms}$。

Table 8: 各負載因子在 S3/S4/S6 故障場景下的極小極大遺憾（ms）。

策略	L=0.80	L=0.85	L=0.88	L=0.90
Prism	7.1	90.0	26.6	**34.8**
fixed_iso	1.4	86.3	**18.8**	34.9
lit_blacklist	**0.0**	**0.2**	189.7	1146.5
no_mitig	86.9	88.7	88.8	51.3

SHED ≈ ISO（在 P2C 下）

在所有 16 種測試配置中，SHED（$w_{\min} = 0.05$）與完全隔離（$w = 0$）的 P99 差距始終 $< 2\,\text{ms}$。相比之下，無緩解與任一主動策略的差距為 5–70 ms。此一數量級的差異證實偵測才是瓶頸，而非策略選擇。

對照組：加權隨機路由

在加權隨機路由（無佇列深度比較）下，SHED 帶來巨大改善：在 $s = 1.5$–$5\times$ 時為 772–15,697 ms。這證明 SHED 的權重削減是真正有效的機制——正是 P2C 的佇列深度比較使其變得邊際化。此結果驗證了 SHED 與 P2C 為互補關係，而非 SHED 本身無效。

6.5 高負載行為

本節掃描 $L \in \{0.80, 0.85, 0.88, 0.90\}$，涵蓋三個故障場景（S3 風格：2 節點/3×；S4 風格：4 節點/混合；S6 風格：級聯）。

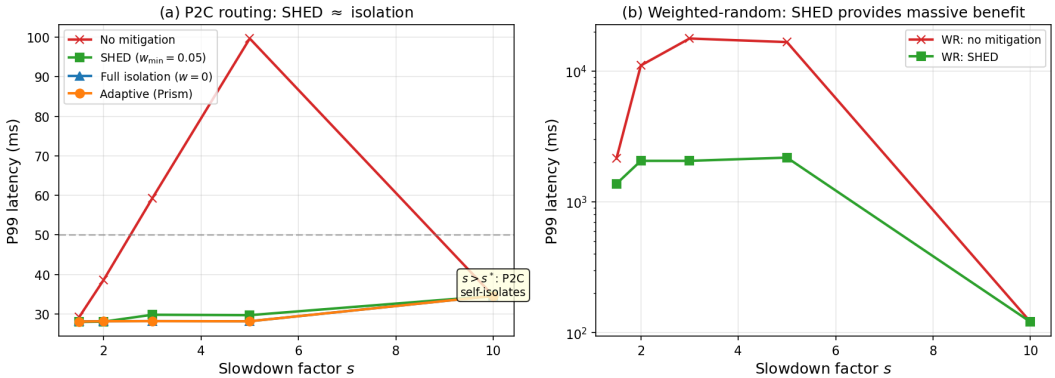

Figure 5: P2C 與加權隨機路由下的嚴重度掃描（$N = 16$，$f = 1$，$L = 0.70$）。(a) 在 P2C 下，SHED 與隔離無法區分；無緩解曲線清楚呈現非單調性。(b) 在加權隨機路由下，SHED 帶來數量級的改善（注意對數刻度），證明 SHED 的權重削減為 P2C 所涵蓋的真實機制。

Table 9: $f = 4$，$N = 32$，$L = 0.80$（無緩解）下的嚴重度非單調性。$s^* = 14.1$（P99）。

s	P99 (ms)	P999 (ms)	p_{fault}	可見性
3×	95.7	118.6	5.20%	可見（> 1%）
5×	185.8	234.7	3.12%	可見
10×	439.0	608.3	1.56%	可見
20×	49.0	17,427	0.78%	不可見（< 1%）

關鍵發現

1. 在 $L = 0.88$ 時，fixed_isolation 的遺憾（18.8 ms）低於 Prism（26.6 ms）；在 $L = 0.90$ 時兩者幾乎打平（34.8 vs. 34.9 ms）。此結果進一步證實互補性定理的預測：在 P2C 下具體策略選擇無關緊要，漸進式回應相較於直接隔離並無系統性優勢。

2. lit_blacklist 在 $L \geq 0.88$ 且 4 個節點故障時出現*災難性失敗*（P99 = 1,407 ms，89% SLO 違反）。根本原因：黑名單完全移除節點，其保守的冷卻期造成振盪（加入黑名單 → 重新整合 → 再次加入黑名單），放大了負載尖峰。

3. 在容量極限（$L = 0.88$，$f = 4$，$\rho_{\text{eff}} = 1.006$），*所有*策略均無法維持 SLO，證實了操作包絡線構成不可逾越的硬限制。

4. 在極端場景下（如所有 8 個節點均發生 2× 故障，$L = 0.80$），主動緩解*反而有害*：no_mitigation（76.3 ms）優於 fixed_isolation（81.7 ms）和 adaptive（95.7 ms）。此時 P2C 已自然地按容量比例分配流量，顯式削減反而使健康節點飽和。此結果進一步證實操作包絡線作為緩解決策前提的必要性。

6.6 嚴重度非單調性驗證

表 9 使用 S8 資料（4 個故障，$N = 32$，$L = 0.80$）驗證臨界嚴重度公式。預測的 $s^* = 14.1$（P99）正確標定了可見性的轉折點：10×（P99 = 439 ms，可見）vs. 20×（P99 = 49 ms，不可見）。流量模型（公式 10）在 $f = 8$ 時精確至三位有效數字，$f = 4$ 時系統性低估 10–14%（見表 3）。

20× 的情況值得特別關注：P99 回到近乎健康的水準（49 ms），但 P999 飆升至 17 秒，因為 $s^*_{P999} = 142.7$（遠高於 20）。故障是「沉默殺手」，對 P99 監控不可見，但於更高百分位造成毀滅性影響。

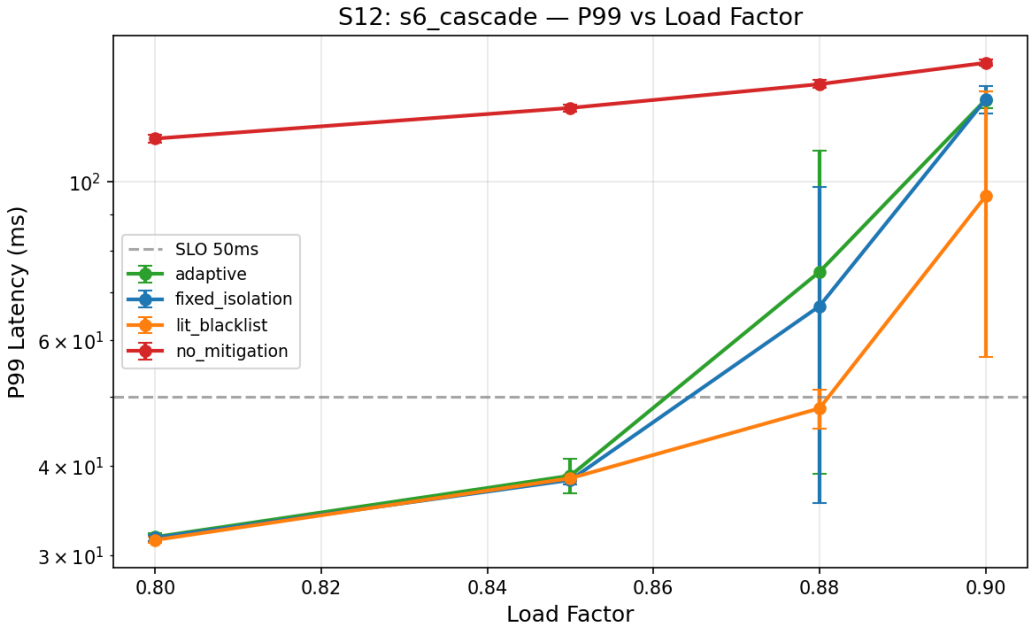

Figure 6: 級聯場景（S6）之 P99 延遲與負載因子關係。所有主動策略在 $L < 0.85$ 時收斂；在 $L \geq 0.88$ 時操作包絡線遭突破，效能急劇下降。lit_blacklist 的振盪使其在高負載時與其他主動策略分離。

6.7 負載均衡策略之影響

進一步的問題是：更精確的負載均衡器能否取代顯式緩解？Least-Outstanding-Requests（LOR）將每個請求路由至 *所有*節點中佇列最短者，理論上優於 P2C 的雙樣本近似。以下在三個故障場景下比較兩種策略。

無緩解時，LOR 在所有場景均領先 P2C 約 30–45 ms：全資訊路由天然避開慢節點。然而，當隔離將故障節點自路由池中移除後，S4（4 個故障）的結果出現反轉：LOR 的 P99 從 P2C 下的 37.8 ms 飆升至 104.4 ms。根本原因是*羊群效應*：LOR 總是將所有並行請求路由至佇列最短的同一節點，造成同步突發，進而使排隊延遲激增。P2C 的隨機雙樣本選取提供了天然的抖動，防止了此同步現象，使其在容量受限的隔離情境下更為穩健。

由此可見，P2C 雖非理論最優卻能成為生產主流，正因其隨機性並非弱點，反而是隔離場景下實現容錯穩健性的關鍵。

6.8 參數敏感度

將所有閾值擾動 ±50%，藉此驗證 Prism 並未過度擬合。在 13 種配置與 6 個場景的組合下，所有配置均達成 P99 < 50 ms。參數空間中不存在「懸崖效應」：效能隨參數偏移而平滑遞減，確認本框架對參數調校的精確度不敏感。

7 相關工作

尾延遲緩解

Dean 和 Barroso [3] 提出對沖請求與綁定請求，至今仍是大規模資料中心系統中應對尾延遲的主要技術。後續研究改進並擴展了這些方法：Vulimiri 等人 [10] 分析了各種延遲分布下的冗餘策略；

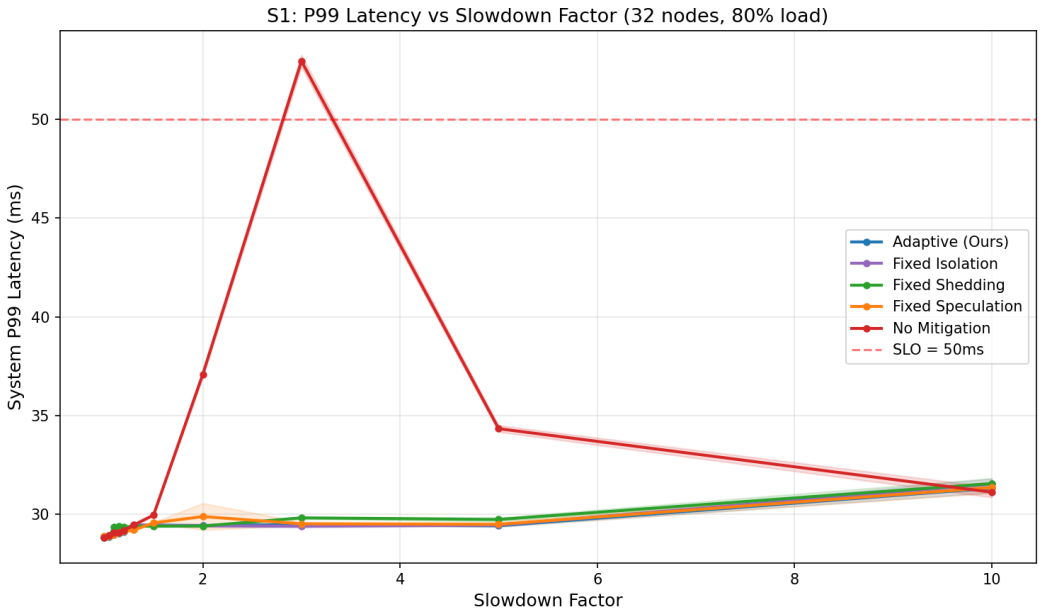

Figure 7: P99 延遲 vs. 減速因子（$N = 32$，$f = 2$，$L = 0.80$）。無緩解曲線呈現明確的非單調性：P99 在 $s = 3$ 附近達到峰值，隨後隨 P2C 自行隔離而下降。所有主動策略在整個嚴重度範圍內維持平坦的 P99。

Ananthanarayanan 等人 [2] 將推測式執行（speculative execution）應用於資料分析中的落後任務（stragglers）。本研究指出，這些反應式機制係針對隨機性慢請求而設計，面對持續性慢故障成效有限——持續性慢故障性質不同，須藉主動偵測方能因應。

灰色故障偵測

Huang 等人 [5] 指出灰色故障的核心在於「觀察者視角不同，可見性亦不同」，並提出利用元件間關聯性加以偵測的方法。Gunawi 等人 [4] 編目了雲端系統中的故障模式，發現效能退化佔相當比例。本研究提出的離開間隔偵測延續此方向，提供不受負載影響的訊號，藉此區分真正的服務減速與因排隊效應所致的延遲上升。

異常值偵測與黑名單

Cassandra 的動態偵測器 [6] 採用指數衰減的延遲評分來迴避慢速副本。Netflix 的自適應路由 [8] 藉由即時異常值偵測進一步延伸此做法。本研究顯示，黑名單機制在中度負載下有效，但在高負載下因移除節點的容量代價而引發災難性振盪。

負載均衡理論

Mitzenmacher [7] 證明在同質伺服器上，P2C 相較於隨機路由可達成指數級的改善。本研究將此分析推廣至異質伺服器情境（即包含不同服務率的故障節點），並推導出嚴重度非單調性現象背後的流量依容量比例分配機制。

Table 10: P2C 與 LOR 負載均衡下的 P99 延遲（ms）（$N = 32$，$L = 0.80$）。

場景	策略	P2C	LOR	Δ
S2（漸進式）	no_mitigation	129.7	84.3	−45.4
	fixed_isolation	30.8	20.7	−10.2
	adaptive	30.8	19.9	−11.0
S4（多節點）	no_mitigation	95.9	59.2	−36.8
	fixed_isolation	37.8	104.4	+66.7
	adaptive	37.7	51.2	+13.5
S6（級聯）	no_mitigation	126.9	83.3	−43.6
	fixed_isolation	32.8	20.3	−12.6
	adaptive	32.8	19.9	−12.9

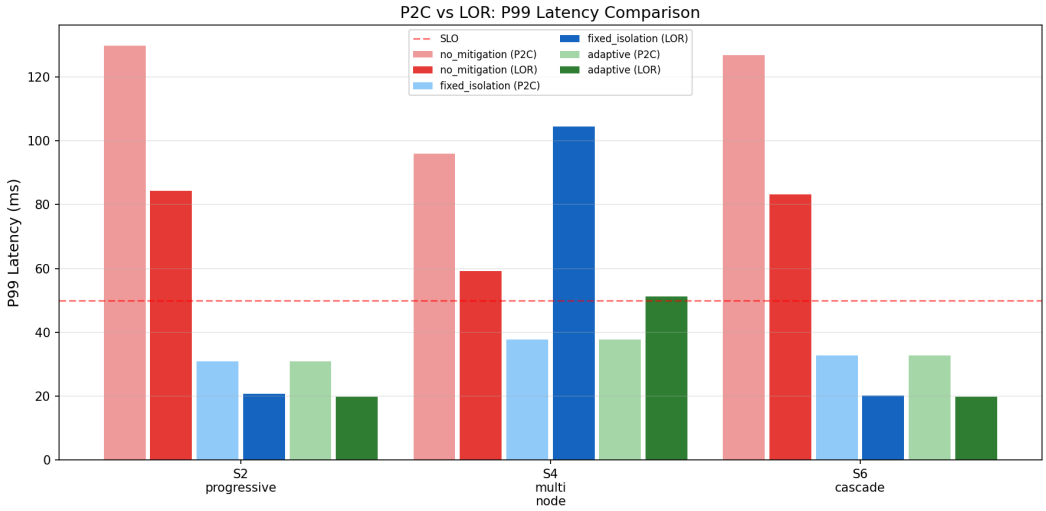

Figure 8: 三個故障場景下 P2C 與 LOR 之比較（$N = 32$，$L = 0.80$）。無緩解時 LOR 優於 P2C，但在 S4（4 個故障）的隔離下遭受羊群效應：LOR+ 隔離的 P99 飆至 104 ms，而 P2C 下僅為 38 ms。

故障下的容量規劃

$N + k$ 冗餘模型已是業界常規，然而在慢故障情境下，負載因子、故障比例與每節點利用率上限三者之間的關聯至今缺乏形式化分析。本研究提出的操作包絡線公式 $f_{\max} = 1 - L/U$ 填補了此一空缺，提供簡潔且經驗證的分析工具。

自適應負載削減

DAGOR [11] 和 CoDel [9] 透過自適應削減應對過載，但主要針對整體系統負載，未涉及逐節點的故障隔離。本研究表明，在佇列感知路由下，具體的回應策略（漸進式削減或完全隔離）差異可忽略，偵測速度才是關鍵瓶頸。此觀點有別於上述側重回應策略設計的研究。

8 結論

本研究從容量理論角度分析 P2C 負載均衡下的慢故障緩解，得出三項主要結論。*操作包絡線*（$f_{\max} = 1 - L/U$）決定緩解何時可行。*臨界嚴重度*（$s^* = fq/((1-q)(N-f))$）識別出哪些減速

區間可由 P2C 的隱式迴避處理，哪些則需要顯式偵測。*SHED–P2C 互補性*（命題 5）表明，在佇列感知路由下，偵測速度比策略選擇更為關鍵：漸進式權重削減與完全隔離達到幾乎相同的 P99，且任何顯式緩解的效益隨叢集大小以 $O(N^{-1.3})$ 衰減。

透過涵蓋 15 個故障場景的評估，本研究歸納出兩項具實務指導意義的發現：(1) 反應式機制（對沖請求、超時重試）對持續性慢故障無效；(2) 主動偵測與流量排除既必要又充分，且在 P2C 下具體的排除機制並不重要。

嚴重度非單調性在實務上意義重大：僅追蹤 P99 延遲的監控面板將無法偵測 $s > s^*$ 的故障。此類故障的 P99 延遲表現正常，卻在 P999 造成災難性影響。系統應監控多個百分位以偵測這些「沉默殺手」。

局限性

本研究存在若干局限：模型假設為單層架構，各工作節點同質且請求之間無親和性約束。多層系統的扇出會放大慢故障影響 [3]，預期操作包絡線將因此收緊。利用率上限 U 由平均延遲推導（推論 2）；對本文所使用的 M/D/1 模型而言，此公式為保守估計。對於高變異服務時間（$C_s^2 \geq 1$），基於平均值的 U 則為樂觀估計（備註 3）。SHED–P2C 互補性的結論以 P2C 路由為前提。在加權隨機等非佇列感知路由下，SHED 仍能帶來顯著效益（表 4），故此結論不可直接推廣至其他路由架構。

未來工作

未來可朝以下兩個方向延伸。第一，*重試放大*：慢故障下的重試會使有效負載增加為 $L_{\text{eff}} = L \cdot (1 + r \cdot p_{\text{timeout}})$，可能縮減操作包絡線。將重試與容量極限的交互作用形式化，對具有重試策略的系統至關重要。第二，將分析擴展至*扇出架構*，其中單一慢節點經由散集-聚集影響 $O(N)$ 個請求，容錯能力可能降至 $f_{\max} \approx (1 - L/U)/\text{fan-out}$。

References

[1] Envoy proxy. `https://www.envoyproxy.io/`. Accessed: 2024.

[2] Ganesh Ananthanarayanan, Ali Ghodsi, Scott Shenker, and Ion Stoica. Effective straggler mitigation: Attack of the clones. In *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 185–198, 2013.

[3] Jeffrey Dean and Luiz André Barroso. The tail at scale. volume 56, pages 74–80. ACM, 2013.

[4] Haryadi S Gunawi, Mingzhe Hao, Riza O Suminto, Agung Laksono, Anang D Satria, Jeffry Adityatama, and Kurnia J Eliber. Fail-slow at scale: Evidence of hardware performance faults in large production systems. In *Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST)*, pages 1–14, 2018.

[5] Peng Huang, Chuanxiong Guo, Lidong Zhou, Jacob R Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. Gray failure: The achilles' heel of cloud-scale systems. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*, pages 150–155, 2017.

[6] Avinash Lakshman and Prashant Malik. Cassandra: A decentralized structured storage system. volume 44, pages 35–40, 2010.

[7] Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104, 2001.

[8] Netflix Technology Blog. Zuul 2: The netflix journey to asynchronous, non-blocking systems. `https://netflixtechblog.com/`, 2020.

[9] Kathleen Nichols and Van Jacobson. Controlling queue delay. volume 10, pages 20–34, 2012.

[10] Ashish Vulimiri, Philip Brighten Godfrey, Radhika Mittal, Justine Sherry, Sylvia Ratnasamy, and Scott Shenker. Low latency via redundancy. In *Proceedings of the 9th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, pages 283–294, 2013.

[11] Yuxin Zhang et al. DAGOR: Overload control for microservices. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2021.